

FIȘA 1. EFICIENȚA ALGORITMILOR ȘI TESTAREA SOLUȚIILOR

Disciplina: Informatică | Clasa a IX-a | Limbaj utilizat: Python

Conținuturi vizate	Eficiența algoritmilor: spațiu de memorie, timp de executare, ordin de complexitate, notația O.
Activități vizate	Evaluarea timpului de executare pentru algoritmi diferiți și compararea a doi algoritmi pentru aceeași problemă.
Obiective	Elevul estimează eficiența unei soluții, identifică numărul de pași importanți și alege varianta mai potrivită pentru un set de date mare.
Durata recomandată	50 minute

1. Reținem ideea principală

Un algoritm este eficient dacă rezolvă corect problema folosind un timp de executare și un spațiu de memorie rezonabile. La clasa a IX-a nu calculăm mereu timpul exact în secunde, ci estimăm cum crește numărul de operații când datele de intrare devin mai mari.

- Timpul de executare se referă la numărul aproximativ de pași importanți efectuați de algoritm.
- Spațiul de memorie se referă la câte variabile sau structuri suplimentare sunt folosite.
- Notația O descrie ordinul de creștere al numărului de operații. De exemplu, $O(n)$ înseamnă că numărul de pași crește aproximativ proporțional cu n .
- O soluție mai rapidă pentru n mic nu este întotdeauna cea mai bună pentru n foarte mare; de aceea testăm și comparăm.

2. Exemplu rezolvat: determinarea divizorilor unui număr

Problema: se citește un număr natural n . Trebuie afișați divizorii săi. Comparăm două variante.

Varianta	Idee	Număr aproximativ de verificări
A	încercăm toți posibii divizori de la 1 la n	n verificări
B	încercăm doar valorile de la 1 la radical din n și afișăm perechea de divizori	aprox. radical din n verificări

```
# Varianta A: parcurgere completă
n = int(input("n = "))
for d in range(1, n + 1):
    if n % d == 0:
        print(d)

# Varianta B: parcurgere până la radical din n
n = int(input("n = "))
d = 1
while d * d <= n:
    if n % d == 0:
        print(d)
        if d != n // d:
            print(n // d)
    d = d + 1
```

Pentru $n = 1000000$, varianta A face 1000000 verificări, iar varianta B face aproximativ 1000 verificări. Varianta B este mai eficientă ca timp, deoarece evită verificări inutile.

3. Exerciții diversificate

A. Completează spațiile libere

1. Un algoritm corect este un algoritm care produce rezultatul _____ pentru toate datele de intrare valide.
2. Notația O este folosită pentru a descrie ordinul de _____ al numărului de operații.
3. Dacă un algoritm parcurge o listă cu n elemente o singură dată, complexitatea este de ordin _____.
4. Un algoritm care folosește o listă suplimentară de n elemente consumă spațiu de memorie de ordin _____.

B. Adevărat sau fals

5. Doi algoritmi care rezolvă aceeași problemă pot avea eficiențe diferite. A / F
6. Un algoritm $O(n^2)$ este întotdeauna mai rapid decât un algoritm $O(n)$. A / F
7. Pentru determinarea divizorilor, este suficient să căutăm până la radical din n dacă afișăm și divizorul pereche. A / F
8. Testarea unui program se face doar cu un singur exemplu, dacă exemplul este corect. A / F

C. Exerciții de lucru

9. Pentru $n = 36$, scrie manual ce divizori găsește varianta B și în ce perechi apar.
10. Compară doi algoritmi care calculează suma elementelor unei liste: unul cu for și unul cu sum(lista). Ce avantaj are fiecare din punctul de vedere al clarității?
11. Scrie un program care măsoară aproximativ timpul de executare pentru căutarea divizorilor prin varianta A și varianta B, folosind modulul time.
12. Pentru următorii algoritmi, indică ordinul de complexitate: a) afișarea primului element din listă; b) parcurgerea întregii liste; c) două parcurgeri imbricate ale aceleiași liste.

```
# Cod lacunar: completează spațiile marcate cu ____
import time
```

```
n = int(input("n = "))
start = time.time()
```

```
for d in range(1, ____ + 1):
    if n % d == 0:
        print(d)
```

```
stop = time.time()
print("Timp:", ____ - start)
```

Barem orientativ / indicații

- A: corect, creștere, $O(n)$, $O(n)$.
- B: 1-A, 2-F, 3-A, 4-F.
- La codul lacunar: range(1, $n + 1$), respectiv stop - start.
- Complexități: acces la primul element $O(1)$, parcurgere $O(n)$, două parcurgeri imbricate $O(n^2)$.